CS410: Parallel Computing

Spring 2025

Nikhil Hegde
Milind Chabbi

Shared-Memory Programming (using POSIX Threads)

Recall: Why Threads?

- Portable, widely-available programming model
  - —use on both serial and parallel systems
- Useful for hiding latency
  - —e.g. latency due to I/O, communication
- Useful for scheduling and load balancing
  - —especially for dynamic concurrency
- Relatively easy to program
  - —significantly easier than message-passing!

Recall:

Shared Address Space Programming Models

A brief taxonomy

- Lightweight processes and threads
  - —all memory is global
  - —examples: Pthreads, Cilk (lazy, lightweight threads)
- Process-based models
  - —each process's data is private, unless otherwise specified
  - —example: Linux shget/shmat/shmdt
- Directive-based models, e.g., OpenMP
  - —shared and private data
  - —facilitate thread creation and synchronization
- Global address space programming languages
  - —shared and private data
  - —hardware typically distributed memory, perhaps not shared
  - —examples: Unified Parallel C, Co-array Fortran

Why Pthreads?

- Lightweight

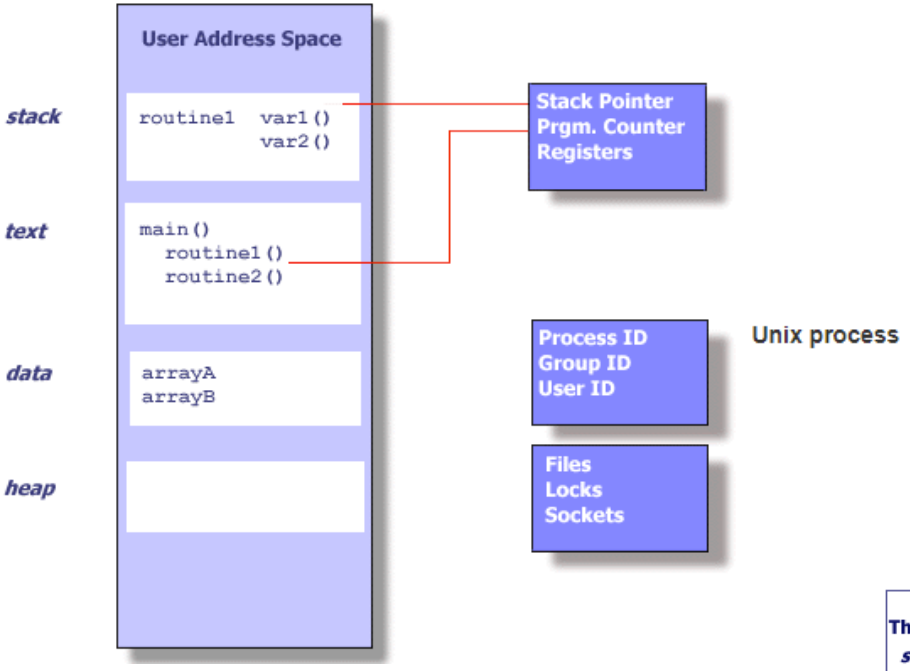
Platform	fork()			pthread_create()		
	real	user	sys	real	user	sys
Intel 2.6 GHz Xeon E5-2670 (16 cores/node)	8.1	0.1	2.9	0.9	0.2	0.3
Intel 2.8 GHz Xeon 5660 (12 cores/node)	4.4	0.4	4.3	0.7	0.2	0.5
AMD 2.3 GHz Opteron (16 cores/node)	12.5	1.0	12.5	1.2	0.2	1.3
AMD 2.4 GHz Opteron (8 cores/node)	17.6	2.2	15.7	1.4	0.3	1.3
IBM 4.0 GHz POWER6 (8 cpus/node)	9.5	0.6	8.8	1.6	0.1	0.4
IBM 1.9 GHz POWER5 p5-575 (8 cpus/node)	64.2	30.7	27.6	1.7	0.6	1.1
IBM 1.5 GHz POWER4 (8 cpus/node)	104.5	48.6	47.2	2.1	1.0	1.5
INTEL 2.4 GHz Xeon (2 cpus/node)	54.9	1.5	20.8	1.6	0.7	0.9
INTEL 1.4 GHz Itanium2 (4 cpus/node)	54.5	1.1	22.2	2.0	1.2	0.6

Source: Why Pthreads? | LLNL HPC Tutorials

Numbers are in seconds and are obtained with 50,000 process/thread creation

Recap: Threads and Processes

Source: what is a Thread? | LLNL HPC Tutorials

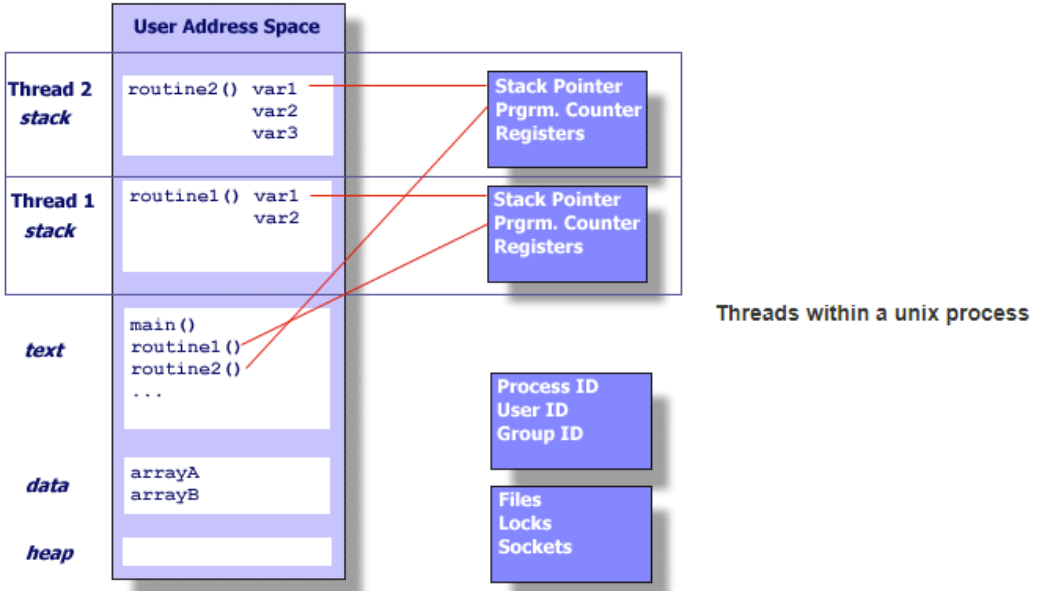


Each process maintains its own:

- Process ID, Group ID, User ID,
- Environment,
- Working directory,
- Registers, stack, heap, program instructions
- File handles, Shared libraries, ..

Each thread in a process maintains its own copy of:

- Stack pointer,
- Program counter,
- Registers,
- Thread-specific data,
- Scheduling policy



POSIX Threads

- Hardware vendors implemented proprietary versions of threads earlier
  - Software portability was a concern

- IEEE POSIX 1003.1c standard
  - Standardized C language threads programming interface for UNIX like systems - **Pthreads API**

  - Implemented as a set of C language programming types and procedure calls
    - Include a header file `pthread.h` and use a library (linking may be implicit in some cases)
    - Fortran Programmers can use wrappers around C procedure calls. Some Fortran compilers provide Fortran Pthreads API.

POSIX Threads

- Pthreads implementation now supported by most vendors
  - In addition to proprietary implementation
- Concepts are broadly applicable to other implementations (independent of Pthreads API)
  - NT Threads
  - Solaris Threads
  - Java Threads
  - C++ Threads (C++11 onwards)

Pthreads API

- Subroutines grouped into four major categories:
  1. **Thread Management –**

     Routines that deal with thread creation, join, detach, etc. Also, query thread attributes
  2. **Mutexes –**

     Routines that deal with synchronization. Creating, destroying, locking, and unlocking a mutex (abbr. for “mutual exclusion”). Also, set mutex attributes
  3. **Condition Variables** –

     Routines addressing communication between threads that share a mutex. Based upon programmer specified conditions. Includes functions to create, destroy, wait and signal based upon specified variable values. Also, set/query condition variable attributes.
  4. **Synchronization**

     Routines that manage read-write locks and barriers

Contains around 100 routines

Pthreads API

- Subroutines grouped into four major categories:
  1. **Thread Management –**

     Routines that deal with thread creation, join, detach, etc. Also, query thread attributes
  2. **Mutexes –**

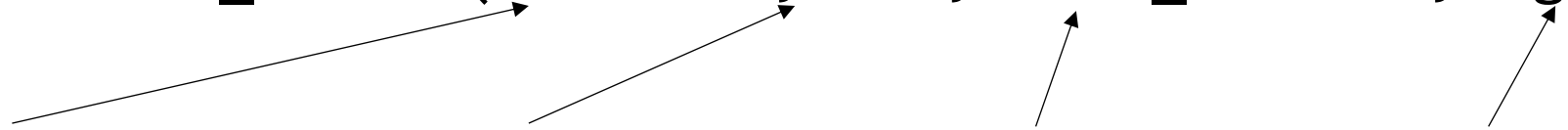
     Routines that deal with synchronization. Creating, destroying, locking, and unlocking a mutex (abbr. for “mutual exclusion”). Also, set mutex attributes
  3. **Condition Variables** –

     Routines addressing communication between threads that share a mutex. Based upon programmer specified conditions. Includes functions to create, destroy, wait and signal based upon specified variable values. Also, set/query condition variable attributes.
  4. **Synchronization**

     Routines that manage read-write locks and barriers

Creating Pthreads

```
pthread_create(threadID,attr,start_routine,arg)
```



unique thread ID Returned upon successful creation.

Thread attributes specified with object of type `pthread_attr_t`. Can leave it as NULL to set default attribute values.

The routine that the thread executes once created.

Single argument passed to `start_routine` (type-casted to void *). Can leave it as NULL to omit arguments.

Creates a new thread and makes it ready-to-execute

- Usage: Typically, `main` function first creates threads, which in turn may create other threads. No hierarchy exists among threads; All threads created are peers.
- Return value: an integer indicating status of creation

Pthread ID `pthread_t`

```
int pthread_create(pthread_t* threadID,..
```

- `pthread_t` object is initialized by the `pthread_create` API.

Pthreads Attributes `pthread_attr_t`

```
int pthread_create(pthread_t* threadID,
const pthread_attr_t* attribute,..
```

- `pthread_attr_init` and `pthread_attr_destroy` APIs create and destroy thread attribute object.

  The attribute object contains options to specify:

  - Detached or joinable state
  - Scheduling inheritance
  - Scheduling policy
  - Scheduling priority
  - Scheduling contention scope
  - Stack size
  - Stack address

Start Routine

```
int pthread_create(pthread_t* threadID,
const pthread_attr_t* attribute,
void * (*start_routine)(void*),..
```

- Called upon thread execution

Arguments to Start Routine

```
int pthread_create(pthread_t* threadID,
const pthread_attr_t* attribute,
void * (*start_routine)(void*),
void * arg)
```

- Single argument typecasted to void *
- How do you pass multiple arguments?
  - Pack the arguments in a structure object and pass the pointer to the structure object (cast as void *)
- Since thread start time is non-deterministic, do not pass as arguments data that is modified by other threads

Terminating Pthreads

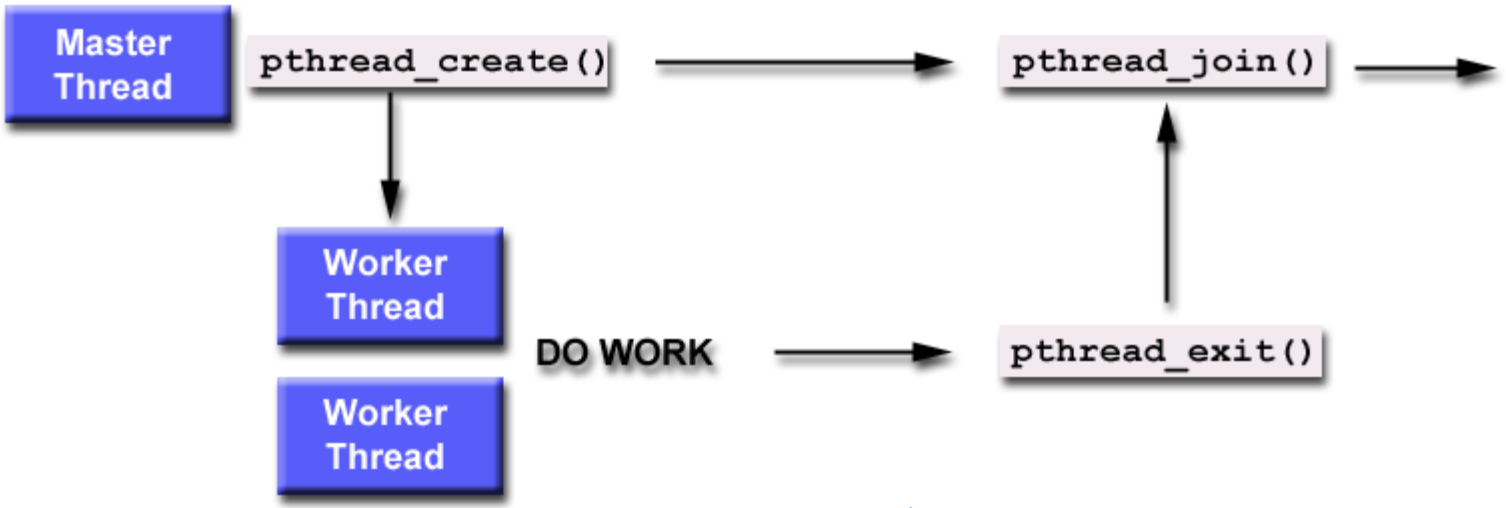
- `pthread_exit(status)`
- `pthread_cancel(thread)` routine can be used by a thread to cancel another thread
- `exit(int)` terminates the entire process (including all threads)

pthread_exit(status)

- `pthread_exit()` routine is called after a thread has completed its work and is no longer required to exist
- If `main()` finishes before the threads it has created, and exits with `pthread_exit()`, the other threads will continue to execute.
  - Otherwise, they will be automatically terminated when `main()` finishes
- The programmer may optionally specify a termination *status*, which is stored as a void pointer for any thread that may join the calling thread
- Cleanup
  - `pthread_exit()` routine does not close files
  - Recommended to use `pthread_exit()` to exit from all threads...especially `main()`.

Joining and Detaching from Pthreads

- `pthread_join(threadid, status)` is one way to accomplish synchronization between threads
  - Blocks the calling thread until the specified thread terminates
  - Only threads created as joinable can be joined (other option is to create a thread as *detached*, where the thread can never be joined.)
    - Can Use `pthread_detach()` to detach a thread after it was created as joinable

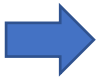


Source: Joining and Detaching Threads | LLNL HPC Tutorials

Pthreads API

- Subroutines grouped into four major categories:
  1. **Thread Management –**

     Routines that deal with thread creation, join, detach, etc. Also, query thread attributes
  2. **Mutexes –**

     Routines that deal with synchronization. Creating, destroying, locking, and unlocking a mutex (abbr. for “mutual exclusion”). Also, set mutex attributes
  3. **Condition Variables** –

     Routines addressing communication between threads that share a mutex. Based upon programmer specified conditions. Includes functions to create, destroy, wait and signal based upon specified variable values. Also, set/query condition variable attributes.
  4. **Synchronization**

     Routines that manage read-write locks and barriers

Mutexes – Protecting Shared Data

- Mutex ("mutual exclusion") – primary means of protecting data when write occurs

An example of a race condition

Thread 1	Thread 2	Balance
Read balance: $1000		$1000
	Read balance: $1000	$1000
	Deposit $200	$1000
Deposit $200		$1000
Update balance $1000+$200		$1200
	Update balance $1000+$200	$1200

Mutex Variables Overview | LLNL HPC Tutorials

- `Balance` must be protected

Mutexes – Protecting Shared Data

- Mutex variable acts like a lock on the shared data
  - Only one Pthread can lock (or acquire / own) a mutex at a time. All other threads wanting to lock must wait till the thread owning the lock unlocks (or releases)
  - What happens when the thread owning the mutex tries to lock the mutex again?
    - Normal – thread deadlocks
    - Recursive – increment a count on the number of times locked, unlock when count reaches zero
    - Errorcheck –
      - report error when thread tries to lock a mutex it has already locked
      - Report error when thread tries to unlock a mutex that some other thread has locked

Pthread Mutex APIs

- Variables are of type `pthread_mutex_t`
- Initialized as:
  - `pthread_mutex_t mymutex = PTHREAD_MUTEX_INITIALIZER;`
  - `pthread_mutex_init(mymutex, attr)`
  - Mutex is unlocked initially
- `pthread_mutex_init(mutex,attr)`
- `pthread_mutex_destroy(mutex)`
- `pthread_mutexattr_init(attr)`
- `pthread_mutexattr_destroy(attr)`
- `pthread_mutex_lock(mutex)`
- `pthread_mutexattr_unlock(mutex)`
- `pthread_mutexattr_trylock(mutex)`

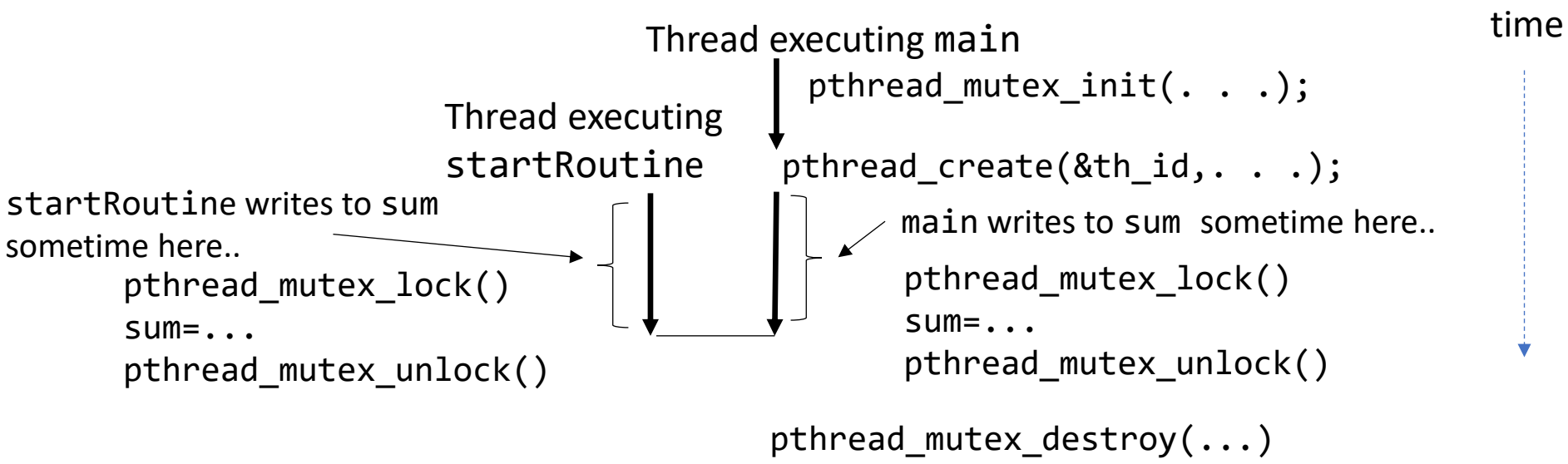
Exercise: When more than one thread is waiting for a locked mutex, which thread will be granted the lock first after the mutex is released?

Synchronization: pthread_mutex_t

```
pthread_mutex_t mymutex;
int sum;
int main() {
 //some code here... (optional)
 pthread_mutex_init(&mymutex, NULL);
 //some code here... (optional)
 pthread_create(&th_id, &myattr, startRoutine, NULL);
//some code here that writes to sum (OR at least
one of startRoutine or main writing to sum)
 pthread_mutex_lock(&mymutex);
 sum=...
 pthread_mutex_unlock(&mymutex);
 //some code here (optional)
 pthread_mutex_destroy(&mymutex);

}
```

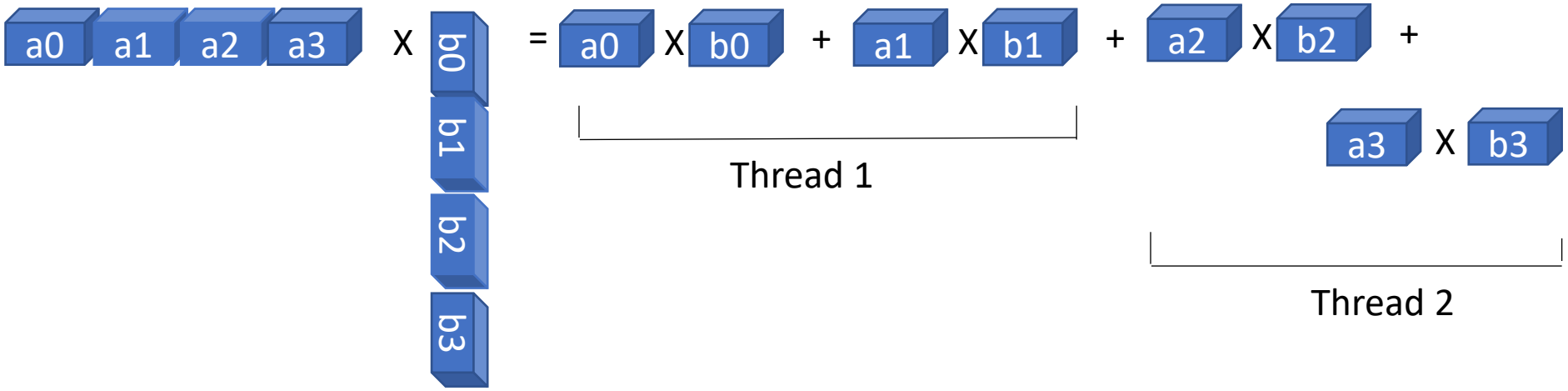
```
int startRoutine() {
...
pthread_mutex_lock(&mymutex);
sum=...
pthread_mutex_unlock(&mymutex);
...

}
```

Synchronization: pthread_mutex_t



Mutex - Example

- Compute Dot Product in Parallel



- Partial result is computed by each thread. Value of the dot product is sum of partial results.
- Demo

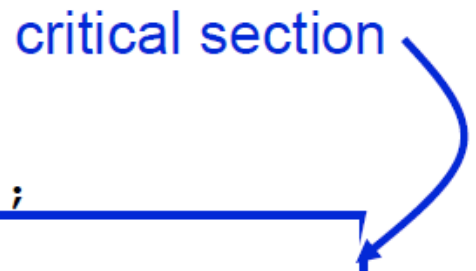
Producer-Consumer Using Mutex Locks

Constraints

- **Producer thread**
  - —must not overwrite the shared buffer until previous task has picked up by a consumer
- **Consumer thread**
  - —must not pick up a task until one is available in the queue
  - —must pick up tasks one at a time

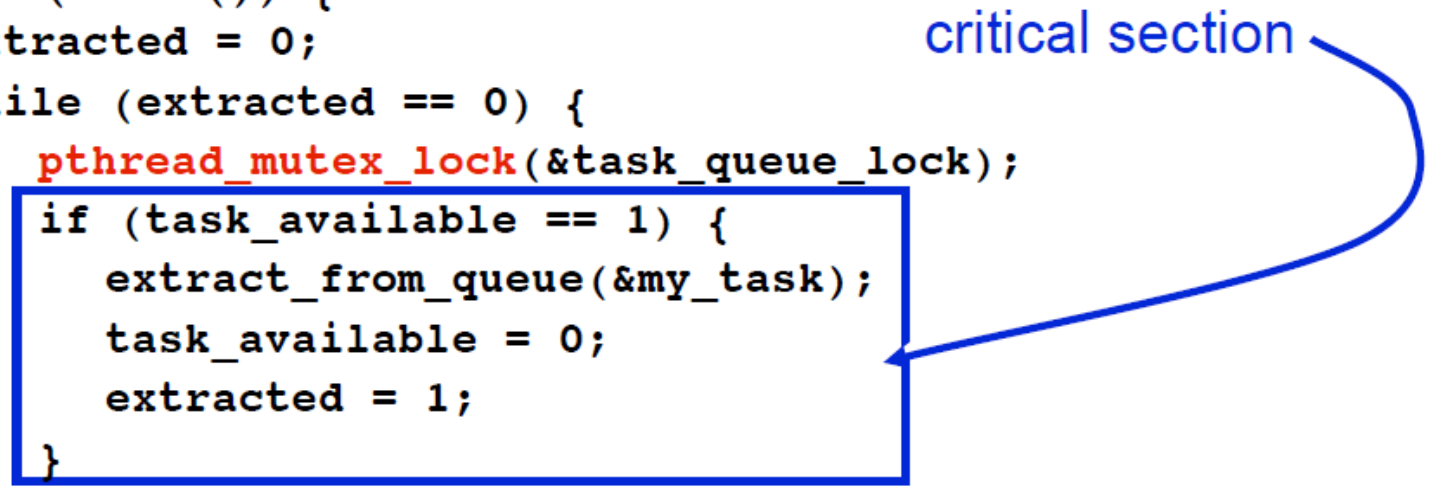
Producer-Consumer Using Mutex Locks

```
pthread_mutex_t task_queue_lock;
int task_available;
...
main() {
    ...
    task_available = 0;
    pthread_mutex_init(&task_queue_lock, NULL);
    ...
}
void *producer(void *producer_thread_data) {
    ...
    while (!done()) {
       inserted = 0;
       create_task(&my_task);
       while (inserted == 0) {
           pthread_mutex_lock(&task_queue_lock);
           if (task_available == 0) {
              insert_into_queue(my_task); task_available = 1;
              inserted = 1;
           }
           pthread_mutex_unlock(&task_queue_lock);
       }
    }
}
```



Producer-Consumer Using Locks

```
void *consumer(void *consumer_thread_data) {
    int extracted;
    struct task my_task;
    /* local data structure declarations */
    while (!done()) {
       extracted = 0;
       while (extracted == 0) {
          pthread_mutex_lock(&task_queue_lock);
          if (task_available == 1) {
             extract_from_queue(&my_task);
             task_available = 0;
             extracted = 1;
          }
          pthread_mutex_unlock(&task_queue_lock);
       }
       process_task(my_task);
    }
```



Overheads of Locking

- Locks enforce serialization
  - —threads must execute critical sections one at a time
- Large critical sections can seriously degrade performance
- Reduce overhead by overlapping computation with waiting

```
int pthread_mutex_trylock(pthread_mutex_t *mutex_lock)
```

  - —acquire lock if available
  - —return EBUSY if not available
  - —enables a thread to do something else if lock unavailable

Pthreads API

- Subroutines grouped into four major categories:
  1. **Thread Management –**

     Routines that deal with thread creation, join, detach, etc. Also, query thread attributes
  2. **Mutexes –**

     Routines that deal with synchronization. Creating, destroying, locking, and unlocking a mutex (abbr. for “mutual exclusion”). Also, set mutex attributes
  3. **Condition Variables** –

     Routines addressing communication between threads that share a mutex. Based upon programmer specified conditions. Includes functions to create, destroy, wait and signal based upon specified variable values. Also, set/query condition variable attributes.
  4. **Synchronization**

     Routines that manage read-write locks and barriers

Condition Variables

- Allows one thread to signal to another thread that the condition is true
- Prevents programmer from looping on a mutex call.
  - Allows to poll if the condition is true

Condition Variables for Synchronization

Condition variable: associated with a predicate and a mutex

- Using a condition variable
  - —thread can block itself until a condition becomes true
    - thread locks a mutex
    - tests a predicate defined on a shared variable
      - if predicate is false, then wait on the condition variable
        - waiting on condition variable unlocks associated mutex
  - —when some thread makes a predicate true
    - that thread can signal the condition variable to either
      - wake one waiting thread
      - wake all waiting threads
    - when thread releases the mutex, it is passed to first waiter

Pthread Condition Variable API

```
/* initialize or destroy a condition variable */
int pthread_cond_init(pthread_cond_t *cond,
    const pthread_condattr_t *attr);
int pthread_cond_destroy(pthread_cond_t *cond);

/* block until a condition is true */
int pthread_cond_wait(pthread_cond_t *cond,
    pthread_mutex_t *mutex);
int pthread_cond_timedwait(pthread_cond_t *cond,
    pthread_mutex_t *mutex,
    const struct timespec *wtime);

/* signal one or all waiting threads that condition is true */
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
```

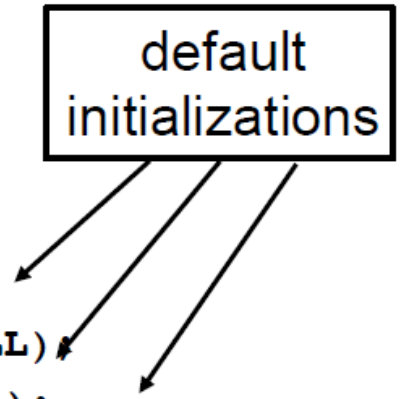
abort wait if time exceeded

wake one

wake all

Condition Variable Producer-Consumer (main)

```
pthread_cond_t cond_queue_empty, cond_queue_full;
pthread_mutex_t task_queue_cond_lock;
int task_available;
/* other data structures here */
main() {
    /* declarations and initializations */
    task_available = 0;
    pthread_init();
    pthread_cond_init(&cond_queue_empty, NULL);
    pthread_cond_init(&cond_queue_full, NULL);
    pthread_mutex_init(&task_queue_cond_lock, NULL);
    /* create and join producer and consumer threads */
}
```



Producer Using Condition Variables

```
void *producer(void *producer_thread_data) {
    int inserted;
    while (!done()) {
       create_task();
       pthread_mutex_lock(&task_queue_cond_lock);
       while (task_available == 1)
          pthread_cond_wait(&cond_queue_empty,
             &task_queue_cond_lock);
       insert_into_queue();
       task_available = 1;
       pthread_cond_signal(&cond_queue_full);
       pthread_mutex_unlock(&task_queue_cond_lock);
    }
}
```

note loop

releases mutex on wait

reacquires mutex when woken

Consumer Using Condition Variables

```
void *consumer(void *consumer_thread_data) {
    while (!done()) {
        pthread_mutex_lock(&task_queue_cond_lock);
        while (task_available == 0)
            pthread_cond_wait(&cond_queue_full,
                &task_queue_cond_lock);
        my_task = extract_from_queue();
        task_available = 0;
        pthread_cond_signal(&cond_queue_empty);
        pthread_mutex_unlock(&task_queue_cond_lock);
        process_task(my_task);
    }
}
```

note loop

releases mutex on wait

reacquires mutex when woken